REST API Routes & HTTP

HTTP methods, status codes, Express routes, and API testing

Agenda

1. **HTTP Methods** – GET, POST, PUT, DELETE
2. **HTTP Status Codes** – communicating outcomes
3. **Express Routes** – building routes in your app
4. **Testing APIs** – Insomnia & Postman

Part 1

HTTP Methods

HTTP Methods

Method	Action	Example
`GET`	Retrieve a resource	`GET /posts/1`
`POST`	Create a new resource	`POST /posts`
`PUT`	Replace a resource	`PUT /posts/1`
`PATCH`	Partially update	`PATCH /posts/1`
`DELETE`	Remove a resource	`DELETE /posts/1`

> The HTTP method tells the server **what action** to perform – use nouns in the URL, not verbs.

GET – Retrieve Resources

Use `GET` to **read** data. It should never modify anything.

```
GET /api/posts       HTTP/1.1   # Get all posts
GET /api/posts/123   HTTP/1.1   # Get a single post
```

- Returns `200 OK` with the resource in the body
- Returns `404 Not Found` if the resource doesn't exist
- **Idempotent** – calling it multiple times has no side effects

POST – Create Resources

Use `POST` to **create** a new resource.

```
POST /api/posts HTTP/1.1
Content-Type: application/json

{
  "title": "Getting Started with TypeScript",
  "content": "TypeScript adds static typing to JavaScript...",
  "author": "Alice Smith"
}
```

- Returns `201 Created` with the new resource
- **Not idempotent** – calling it twice creates two resources

PUT – Replace Resources

Use `PUT` to **fully replace** an existing resource.

```
PUT /api/posts/123 HTTP/1.1
Content-Type: application/json

{
  "title": "Advanced TypeScript Patterns",
  "content": "Exploring generics and utility types...",
  "author": "Alice Smith"
}
```

- Returns `200 OK` or `204 No Content`
- **Idempotent** – same request always produces the same result
- Replaces the **entire** resource, not just some fields

DELETE – Remove Resources

Use `DELETE` to **remove** a resource.

```
DELETE /api/posts/123 HTTP/1.1
```

- Returns `204 No Content` on success (no body needed)
- Returns `404 Not Found` if the resource doesn't exist
- **Idempotent** – deleting something that's already gone is still a success

Idempotency

Idempotent – the same request can be made multiple times with the same result.

Method	Idempotent?	Why?
`GET`	✅ Yes	Only reads data
`POST`	❌ No	Creates a new resource each time
`PUT`	✅ Yes	Replaces with the same data
`DELETE`	✅ Yes	Resource is gone either way

> This matters for **retries** – if a network request fails, is it safe to retry?

Part 2
HTTP Status Codes

Status Code Groups

Range	Category	Meaning
`1xx`	Informational	Request received, continuing
`2xx`	Success	Request completed successfully
`3xx`	Redirection	Further action needed
`4xx`	Client Error	Problem with the request
`5xx`	Server Error	Problem on the server

> Always return the **most specific** status code that applies.

2xx – Success Codes

Code	Name	When to use
`200 OK`	Generic success	`GET`, `PUT`, `PATCH`
`201 Created`	Resource created	`POST` – include `Location` header
`204 No Content`	Success, no body	`DELETE`, `PUT` with no response body
`202 Accepted`	Async processing	Long-running background operations

```
HTTP/1.1 201 Created
Location: /api/posts/42
Content-Type: application/json

{ "id": 42, "title": "Getting Started with TypeScript", "content": "TypeScript adds static typing ... ", "author": "Alice
```

4xx – Client Error Codes

Code	Name	When to use
`400 Bad Request`	Invalid data	Missing fields, wrong types
`401 Unauthorized`	Not authenticated	No token or invalid token
`403 Forbidden`	Not authorised	Valid token, wrong permissions
`404 Not Found`	Resource missing	ID doesn’t exist
`405 Method Not Allowed`	Wrong method	`DELETE` on a read-only route
`409 Conflict`	State conflict	Duplicate resource

> `401` = "who are you?" · `403` = "I know who you are, but no."

5xx – Server Error Codes

Code	Name	When to use
`500 Internal Server Error`	Unhandled exception	Unexpected server crash
`503 Service Unavailable`	Server overloaded	App is down or restarting

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "error": "An unexpected error occurred",
  "code": "INTERNAL_ERROR"
}
```

> Never expose stack traces or internal details in error responses.

Matching Methods to Status Codes

Method	Success	Common Errors
`GET`	`200 OK`	`404 Not Found`
`POST`	`201 Created`	`400 Bad Request`
`PUT`	`200 OK` / `204`	`404`, `409 Conflict`
`DELETE`	`204 No Content`	`404 Not Found`

Part 3

Express Routes

Basic Route Structure

In Express, a route maps an **HTTP method + path** to a handler function.

```
import express from "express";
const router = express.Router();

router.get("/posts", (req, res) => {
  // handle GET /posts
});

router.post("/posts", (req, res) => {
  // handle POST /posts
});

export default router;
```

GET Routes

```javascript
// Get all posts
router.get("/posts", (req, res) => {
  const posts = [
    { id: 1, title: "Getting Started", content: "...", author: "Alice" }
  ];
  res.status(200).json(posts);
});

// Get a single post by ID
router.get("/posts/:id", (req, res) => {
  const { id } = req.params;
  const post = { id: Number(id), title: "Getting Started", author: "Alice" };

  if (!post) {
    return res.status(404).json({ error: "Post not found" });
  }

  res.status(200).json(post);
});
```

POST Route

```
// Create a new post
router.post("/posts", (req, res) => {
  const { title, content, author } = req.body;

  // In a real app, save to database here
  const newPost = {
    id: 2,
    title,
    content,
    author
  };

  res.status(201).json(newPost);
});
```

> Use `req.body` to access the request payload – make sure `express.json()` middleware is applied.

PUT & DELETE Routes

```
// Replace a post
router.put("/posts/:id", (req, res) => {
  const { id } = req.params;
  const { title, content, author } = req.body;

  // In a real app, replace in database here
  const updated = { id: Number(id), title, content, author };
  res.status(200).json(updated);
});

// Delete a post
router.delete("/posts/:id", (req, res) => {
  const { id } = req.params;

  // In a real app, delete from database here
  res.status(204).send();
});
```

Registering Your Router

Wire your router into the Express app in `app.ts`:

```
import express from "express";
import postRouter from "./routes/postRouter";

const app = express();

app.use(express.json()); // Parse JSON request bodies

app.use("/api", postRouter); // Mount routes under /api

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Routes are now available at `/api/posts`.

Route Summary

```
GET     /api/posts         → list all posts
GET     /api/posts/:id     → get a single post
POST    /api/posts         → create a new post
PUT     /api/posts/:id     → replace a post
DELETE  /api/posts/:id     → delete a post
```

Rule: Use nouns, not verbs. The HTTP method **is** the verb. ❌ `GET /api/getPosts` ✅ `GET /api/posts`

Part 4

Testing APIs

Why Use an API Client?

Before connecting a frontend, you need to test your API works correctly.

Insomnia and **Postman** are GUI tools that let you:

- Send HTTP requests to any URL
- Set headers, body, and authentication
- Inspect responses and status codes
- Save requests and organise them into collections
- Share collections with your team

Testing a POST Request

In Insomnia or Postman:

```
Method:  POST
URL:     http://localhost:3000/api/posts
Headers: Content-Type: application/json
Body:
{
  "title": "My First Blog Post",
  "content": "This is an example post...",
  "author": "Your Name"
}
```

Expected response:

```
HTTP 201 Created
{ "id": 2, "title": "My First Blog Post", "content": "This is an example post...", "author": "Your Name" }
```

What to Check When Testing

Check	What to look for
Status code	Is it the correct code for the operation?
Response body	Is the data what you expected?
Headers	Is `Content-Type: application/json` set?
Error handling	Does a bad request return `400`?
404 handling	Does a missing ID return `404`?

Key Takeaways

- Use **HTTP methods** to express the action: `GET`, `POST`, `PUT`, `DELETE`
- Use **status codes** to communicate the outcome precisely
- Use `req.params` for route parameters (`:id`), `req.body` for request payloads
- Always mount routes under a prefix (e.g. `/api`)
- Test every route with **Insomnia or Postman** before wiring up a frontend

Time to Code

Exercise 2 – Add Routes to Your App

Create routes for:

- Get all items in your chosen domain
- Get a single item by ID
- Create a new item
- Update an item
- Delete an item